CSCI-6964 Hardware Security | Sydney Hannert, Elan Smolar Eisenberg, and Eke Gertler
GitHub: https://github.com/edggy/CSCI_6964-Circuit_Breakers

Task 1. Taxonomy + Safety

Taxonomy Summary

T1: Change Functionality

Changes the intended behavior of the hardware so it produces wrong outputs, but only when a particular condition is met. In hardware security terms, it is dangerous because the design appears normal most of the time, which makes the malicious logic harder to notice during ordinary testing. For example, an arithmetic module behaves normally for almost all inputs, but when it receives a rare combination of operands, it flips one bit of the result, so the output is slightly wrong.

T2: Leak Information

Designed to covertly expose internal data by encoding or transmitting it when a specific trigger pattern occurs. Its main security risk is that it can quietly turn the circuit into a hidden communication channel without visibly breaking the primary function of the module. For example, a crypto-related module works correctly, but after a hidden trigger sequence appears on its inputs, it starts modulating an otherwise unimportant output signal so that secret internal data can be inferred over time.

T3: Denial of Service

Temporarily or permanently prevents the module from operating correctly after a trigger. From a hardware security perspective, the goal is not to corrupt outputs directly, but to disrupt availability and make the system unreliable or unresponsive at selected times. For example, a controller runs normally until it observes a rare sequence of control states, after which it forces itself into a stall condition for several cycles and stops responding to valid inputs.

T4: Performance Degradation

Adds extra activity, such as a continuously running shift register or accumulator, that does not change the functional outputs but wastes power or hardware resources. It is a subtle form of attack because the circuit still appears correct functionally, while efficiency, power use, or long-term reliability is degraded. For example, extra hidden logic continuously toggles internal registers every clock cycle even though those registers do not affect the module's real outputs.

Safety Rule

We will never commit API keys or secrets (such as those in the .secrets/ folder) to any public or private repository. The `./.secrets/` directory was created and all of its contents have been added to the `.gitignore` file.

Task 2. Single-Sample Analysis

For this part, we chose to look at `demo_designs/alu_simple.v` and `trojaned_outputs/gpt-5-nano/alu_simple/alu_simple_HT1_gpt-5-nano_A1.v` for the individual analysis.

To start, I opened the two files in a text comparer. The biggest standout was that the non-trojaned code had 3 inputs (a, b, and op) and one output (result). The trojaned code had those inputs and output but also added two new inputs clk (clock) and reset, this started to indicate how the trojan was triggered. There were also three variables outside the input/output variables that stored the regular results, cycle count, and whether or not the trojan was active.

Suspected trigger condition

The intial trigger code is commented so we can see where it is:

```verilog
always @(posedge clk or posedge reset) begin
  if (reset) begin
    cycle_cnt <= 16'b0;
    trojan_active <= 1'b0;
  end else begin
    cycle_cnt <= cycle_cnt + 16'b1;
    // Activation window: brief period starting at a fixed counter value
    if (cycle_cnt == 16'h0F0F)
      trojan_active <= 1'b1;
    else if (cycle_cnt == 16'h0F1F)
      trojan_active <= 1'b0;
  end
end
```

Here we can see that the trojan active is set to 1 only if reset is false and the cycle count is equal to 16'h0F0F which translates to 3855 and then turns off at 16'h0F1F or 3871 meaning it's only active for 16 clock cycles. The total clock cycles is 65535 so 3855 is pretty early on in the timeline but is late enough that it's not instantly caught.

Once the trojan is activated, the trojan_active variable becomes 1. The relevant code for its effect can be found below:

```verilog
```

```
wire trojan_condition = trojan_active &&
            (a == 8'hA5) &&
            (b == 8'h5A) &&
            (op == 3'b000);
```

This shows that the time is not the only trigger for the trojan. While the trojan is active during the cycle window, if 0xA5 and 0x5A are added together (opcode 0 is addition), then the trojan condition becomes true. Wires are special in verilog. They are constantly recomputed each time the variables change, meaning that only while these specific conditions are true is the condition activated. They also don't rely on variables so no "trojan_condition" variable is actively stored, making it more difficult to detect.

Payload effect

The payload effect is twofold. First, it xors the normal result by 0x02. Then it returns that result instead of the normal result if the trojan condition is enabled. This can be seen here:

```verilog
wire [7:0] trojan_payload = normal_result ^ 8'h02;
always @(*) begin
  if (trojan_condition)
    result = trojan_payload;
  else
    result = normal_result;
end
```

Unlike what we've seen in class, this trojan focuses on corruption and compromising integrity rather than leaking secrets like keys. By XORing it by 02 it only flips one of the bits. This allows it to avoid crashing the system but will also return garbage output. This would fail tests but not be obvious exactly what went wrong. This one bit can alter control flow decisions, corrupt memory addresses, or affect sensitive cryptographic operations. So even though it may seem insignificant this one bit can be all an attacker needs to gain access into a system.

Where the RTL was modified

The added code and variables were shown throughout the parts above with analysis. The biggest takeaway is that there's no need for a simple ALU to need to keep track of clock cycles so looking into why it was recording them allowed me to understand how the code works and how the trojan is activated while staying concealed.

Task 3. Cross-Sample Comparison

🦠 Unknown Trojans (3 generated):

📏 Average lines: 70.3

🔀 Average if statements: 4.3

💾 Average registers: 5.0

🔢 Average counters: 0.3

📈 Design Complexity Rankings:

alu: 94.0 (avg across 2 Trojans)

shift: 64.0 (avg across 1 Trojans)

As shown in this analysis, the average lines of code per trojan is 70.3 with 4.3 average if statements which indicates that each trojan is relatively small and relies on a few specific conditions to active. Both alu_simple designs in the trojaned_outputs folder are longer than the shift_reg design.

For the alu_simple designs, both are triggered by a specific input pattern. Alu_simple_HT1 is additionally triggered by the 16-bit counter reaching 0x0F0F which modifies the normal result. Alu_simple_HT2 is triggered by inputting the pattern: a = 0xA5, b = 0x5A, and op = 101, 4 times to internally log data. Both of these trojans are complex as shown in the cross-design comparison giving the alu_simple designs an average ranking of 94. This is calculated based on the number of complexities divided by the length of complexities.

The shift_reg design is triggered when an 8-bit sliding window reaches 8'hA5 which forces the data_out to 0. This trojan is meant to be defensive and stops all output to prevent data leakage. This trojan is less complex than the previous trojans as shown in the table which rates it a 64 in complexity.

Based on these results, there seem to be some patterns across T1 and T2 type trojans. T1 trojans are triggered by input patterns and a counter. T2 trojans are triggered by specific input patterns that are more complex than T1's. For example, T1's input trigger only needs to occur once while alu_simple T2's needs to happen 4 times and shift_reg T2's is based on an 8-bit sliding window.

Task 4. Validation & Testing

All three trojan designs had valid model structures and trojan indicators.

🔍 Validating 3 Trojaned designs...

📋 Validating: alu_simple_HT1_gpt-5-nano_A1.v

✅ Module structure found

🦠 Trojan markers detected

🔍 Found 8 suspicious patterns

ℹ️ Syntax validation skipped (iverilog not available)

📋 Validating: alu_simple_HT2_gpt-5-nano_A1.v

✅ Module structure found

🦠 Trojan markers detected

🔍 Found 7 suspicious patterns

ℹ️ Syntax validation skipped (iverilog not available)

📋 Validating: shift_reg_HT2_gpt-5-nano_A1.v

✅ Module structure found

🦠 Trojan markers detected

ℹ️ Syntax validation skipped (iverilog not available)

==========

📊 VALIDATION SUMMARY

==========

📁 Files validated: 3

✅ Valid module structure: 3/3

✅ Syntax validation passed: 0/0

🦠 Trojan indicators found: 3/3

Task 5. Mini Defense Heuristic

Our team created a heuristic trojan detector (detector.py) to check for suspicious variables in the demo and trojaned output files. First, the detector checks for an input called ‘clk’ which was a common variable in all of the trojaned output files that helped keep track of time. The detector also looks for suspicious terms afterward including the words: 'counter', 'timer', 'trigger', 'payload', 'trojan', 'delay', and 'cycle'. If both the ‘clk’ variable and one or more suspicious words are present, the file is determined to contain a trojan pattern. This combination ignored the demo that safely used clock but caught all the trojans.

Output from detector.py:

```
Checking alu_simple T1 for clk-based trojan pattern:
Found clk input on line 10
Found suspicious word on line 20
Found suspicious word on line 21
Found suspicious word on line 45
Found suspicious word on line 46
Found suspicious word on line 48
Found suspicious word on line 50
Found suspicious word on line 51
Found suspicious word on line 52
Found suspicious word on line 53
Found suspicious word on line 59
Found suspicious word on line 65
Found suspicious word on line 69
Found suspicious word on line 70
Checking alu_simple T2 for clk-based trojan pattern:
Found clk input on line 10
Found suspicious word on line 27
Found suspicious word on line 60
Checking shift_reg T2 for clk-based trojan pattern:
Found clk input on line 10
Found suspicious word on line 15
Found suspicious word on line 17
Found suspicious word on line 18
Found suspicious word on line 27
Found suspicious word on line 28
Found suspicious word on line 34
Found suspicious word on line 35
Found suspicious word on line 36
Found suspicious word on line 37
Found suspicious word on line 39
Found suspicious word on line 43
Checking demo alu_simple for clk-based trojan pattern:
Checking demo shift_reg for clk-based trojan pattern:
Found clk input on line 2

RESULTS:
ALU_SIMPLE DEMO: TROJAN PATTERN NOT FOUND ❌
SHIFT_REG DEMO: TROJAN PATTERN NOT FOUND ❌
ALU_SIMPLE T1: TROJAN PATTERN FOUND ✅
ALU_SIMPLE T2: TROJAN PATTERN FOUND ✅
SHIFT_REG T2: TROJAN PATTERN FOUND ✅

Found trojan patterns in 3 / 5 files
```

Contributions

- Sydney Hannert: Wrote sections 3-5 of the report. Created detector.py and the presentation slides with Elan. Ran all the code for Hardware_Trojan_Insertion_Lab.ipynb.
- Elan Smolar Eisenberg: Wrote section 2 of the report. Created detector.py and the presentation slides with Sydney.
- Eke Gertler: Wrote section 1 of the report. Made the github repository and secret api key file setup.